

TideWAL: Timestamp-Integrated Dependency-Closed Parallel WAL for Main-Memory Transaction Processing

Shun Sasaki¹ Takayuki Tanabe¹ Hideyuki Kawashima¹

概要: This paper proposes TideWAL, a dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL avoids conservative global-prefix durable acknowledgment while preserving recovery safety by acknowledging a transaction only after both its own log record and the durable frontier of its observed dependencies are persistent. In an ERMIA-style implementation, TideWAL reduces pending durable commits from 65,522 to 48 and p99 durable-acknowledgment latency from 131 ms to 512 μ s on YCSB-B at 32 threads, while sustaining 231K durable acknowledgments per second.

1. Introduction

1.1 Motivation

Transaction processing is a core abstraction behind data-intensive applications such as banking, payment processing, inventory management, ticket reservation, and online services that update shared state under high concurrency. A transaction processing system must provide both correct concurrent execution and crash recovery; in this sense, transaction processing consists of concurrency control (CC) and recovery [1]. A large body of work has studied scalable CC for multicore main-memory databases, including Silo, ERMIA, TicToc, SI, SSI, and SSN-style protocols [2], [3], [4], [5], [6], [7]. These protocols improve isolation and scalability by reducing centralized validation, timestamp, and version-management bottlenecks. In contrast, the recovery side of modern CC protocols has been explored less systematically: when a high-performance CC protocol is combined with crash durability, it is often unclear which recovery protocol preserves both safety and scalability.

Write-ahead logging (WAL) is the standard foundation for database recovery, with ARIES being the classical design [8], [9]. Modern systems have revisited WAL for multicore and fast-storage environments. SiloR paral-

lelizes logging, checkpointing, and recovery for Silo; Passive Group Commit reduces synchronization overhead on non-volatile memory; P-WAL partitions the log into multiple streams; and Taurus generalizes parallel logging with dependency tracking [10], [11], [12], [13]. Although these systems differ in the storage device, log format, and recovery procedure they target, they share an important lesson: writing log records in parallel is essential for making durability compatible with multicore transaction processing.

The motivation of this work is to clarify how parallel logging should be combined with strong multiversion CC protocols. SI, SSI, and SSN are powerful because they allow readers and writers to overlap while controlling anomalies through timestamps and dependency information. However, their dependency structure also makes durable acknowledgment more subtle than simply checking whether a local log stream has reached a given position. We therefore study how to combine P-WAL-style parallel logging with SSN, the most dependency-rich protocol among these candidates, and how to acknowledge durable commits without unnecessarily waiting for unrelated log streams.

1.2 Problem

We target SSN because it is the most complex of the SI-family protocols we consider: it certifies serializability using dependency metadata, while the underlying en-

¹ Faculty of Environment and Information Studies, Keio University, Japan

gine still uses timestamped multiversion execution. As the baseline recovery protocol, we start from P-WAL, which is also used as a foundation of Shirakami, the transaction processing mechanism of Tsurugi [12], [14]. P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or update a global durable prefix. This duplicates ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

Second, workers can spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If workers wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U , acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

Figure 2 shows durable-acknowledgment throughput on ERMIA with YCSB-B [15] from 1 to 32 worker threads. Single WAL stays near 12–13K acknowledgments/s, while P-WAL scales to 109K acknowledgments/s by partitioning log streams. TideWAL reaches 374K acknowledgments/s at 32 workers by combining parallel WAL logging with asynchronous dependency-closed durable acknowledgment.

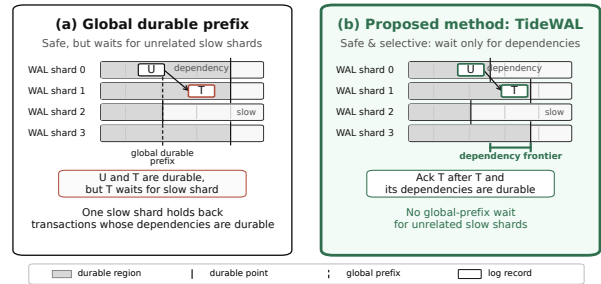


FIG 1 Durable acknowledgment policies in parallel WAL. Global-prefix acknowledgment is safe but can delay transactions whose dependencies are already durable because it waits for unrelated slow shards. TideWAL uses dependency-closed acknowledgment, acknowledging a transaction only after its own log record and dependency frontier are durable.

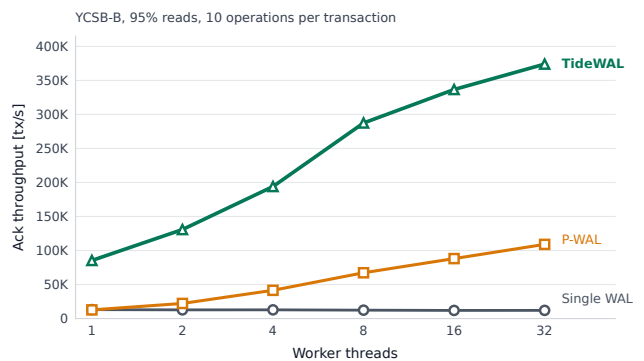


FIG 2 YCSB-B durable-acknowledgment throughput on ERMIA. TideWAL scales beyond Single WAL and P-WAL by combining parallel WAL logging with asynchronous dependency-closed durable acknowledgment.

1.3 Approach

This paper proposes *TideWAL*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL targets timestamp-based engines such as ERMIA, where the transaction engine already computes a logical commit order. Its goal is not merely to maximize raw throughput, but to return durable commit acknowledgments safely without the backlog and tail latency caused by global-prefix waiting.

TideWAL addresses the three problems above with three corresponding techniques.

A1. Commit timestamps as logical LSNs. TideWAL reuses the SSN/ERMIA commit timestamp as the logical LSN for recovery. WAL shards still maintain local physical offsets, but the WAL layer does not allocate an additional global logical order. This removes duplicated global ordering between CC and recovery and reduces global-counter accesses on the commit path.

A2. Worker/flusher/committer separation. TideWAL decouples transaction execution from durable acknowledgment. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local logs. Committers observe durable-frontier advances and return durable acknowledgments. This pipeline keeps workers from directly blocking on `fdatsync` or on durable-prefix notification waits.

A3. Dependency-closed durable acknowledgment. TideWAL replaces global-prefix acknowledgment with dependency-closed acknowledgment. For each transaction, TideWAL maintains a dependency frontier that summarizes the log positions that must be recoverable before the transaction can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This preserves recovery safety while avoiding waits for unrelated WAL shards.

We implement TideWAL in an ERMIA-style engine and evaluate it on YCSB workloads. On YCSB-B at 32 threads, worker-wait global-prefix WAL reaches only 106K durable acknowledgments per second. An asynchronous global-prefix design improves throughput to 365K acknowledgments per second, but at the cost of 65,522 pending commits and 131 ms p99 durable-acknowledgment latency. TideWAL sustains 231K durable acknowledgments per second while reducing pending commits to 48 and p99 latency to 512 μ s. These results show that dependency-closed acknowledgment controls durable-acknowledgment backlog and tail latency while preserving recovery safety on sparse-dependency workloads.

We also evaluate timestamp integration. Under real I/O, storage synchronization dominates, so the difference between separate-LSN logging and TideWAL is small. Under an I/O-light configuration, however, reusing commit timestamps as logical LSNs improves throughput by 7.4% by eliminating WAL-side global LSN allocation. This result shows that TideWAL removes duplicated ordering between concurrency control and durability, and that the benefit becomes visible once the storage bottleneck is weakened.

1.4 Organization

The rest of this paper is organized as follows. Section 2 reviews WAL, parallel WAL, timestamp-based concurrency control, and dependency requirements for recovery. Section 3 presents TideWAL's timestamp-integrated logical ordering, dependency-frontier construction, and

worker/flusher/committer pipeline. Section 4 discusses the correctness of dependency-closed durable acknowledgment. Section 5 evaluates durable-acknowledgment throughput, pending commits, and tail latency. Section 6 discusses related work. Section 7 concludes the paper.

2. Background

3. Design

4. Correctness

5. Evaluation

6. Related Work

7. Conclusion

謝辞

参考文献

- [1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).
- [2] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).
- [3] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).
- [4] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).
- [5] Ports, D. R. K. and Grittnier, K.: Serializable Snapshot Isolation in PostgreSQL, *Proc. VLDB Endow.*, Vol. 5, No. 12, p. 1850–1861 (online), DOI: 10.14778/2367502.2367523 (2012).
- [6] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Efficiently Making (Almost) Any Concurrency Control Mechanism Serializable, *The VLDB Journal*, Vol. 26, No. 4, pp. 537–562 (online), DOI: 10.1007/s00778-017-0463-8 (2017).
- [7] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).
- [8] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

- [9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).
- [10] Zheng, W., Tu, S., Kohler, E. and Liskov, B.: Fast Databases with Fast Durability and Recovery Through Multicore Parallelism, *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '14, USENIX Association, pp. 465–477 (online), available from <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng-wenting> (2014).
- [11] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Scalable Logging Through Emerging Non-Volatile Memory, *Proc. VLDB Endow.*, Vol. 7, No. 10, pp. 865–876 (online), available from <https://www.vldb.org/pvldb/vol7/p865-wang.pdf> (2014).
- [12] 孝明神谷, 英之川島, 喬 星野, 修見建部, Komei, K., Hideyuki, K., Takashi, H., Osamu, T.: P-WAL: Paralell Write-ahead Logging, 情報処理学会論文誌データベース (TOD), Vol. 10, No. 1, pp. 24–39 (online), available from <https://cir.nii.ac.jp/crid/1050282812885062144> (2017).
- [13] Xia, Y., Yu, X., Pavlo, A. and Devadas, S.: Taurus: lightweight parallel logging for in-memory database management systems, *Proc. VLDB Endow.*, Vol. 14, No. 2, p. 189–201 (online), DOI: 10.14778/3425879.3425889 (2020).
- [14] Kawashima, H.: Next-generation Database Tsurugi: Concurrency Control and Recovery Systems, *IPSP Magazine*, Vol. 66, No. 4, pp. e9–e15 (online), DOI: 10.20729/0002001646 (2025).
- [15] Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R. and Sears, R.: Benchmarking cloud serving systems with YCSB, *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, New York, NY, USA, Association for Computing Machinery, p. 143–154 (online), DOI: 10.1145/1807128.1807152 (2010).